

EASY

Q1 - Show first name, last name, and gender of patients whose gender is 'M'

```
1 SELECT first_name, last_name, gender FROM patients
2 WHERE gender = 'M';s|
```

Q2 - Show first name and last name of patients who does not have allergies. (null)

```
1 SELECT first_name, last_name FROM patients
2 WHERE allergies IS NULL;
```

Q3 - Show first name of patients that start with the letter 'C'

```
1 SELECT first_name FROM patients
2 WHERE first_name LIKE "C%";
```

Q4 - Show first name and last name of patients that weight within the range of 100 to 120 (inclusive)

```
1 SELECT first_name, last_name FROM patients
2 WHERE weight BETWEEN 100 AND 120;
```

Q5 - Update the patients table for the allergies column. If the patient's allergies is null then replace it with 'NKA'

```
1 UPDATE patients
2 SET allergies = 'NKA'
3 WHERE allergies IS NULL;
```

Q6 - Show first name and last name concatenated into one column to show their full name.

```
1 SELECT CONCAT(first_name, ' ', last_name) AS full_name
2 FROM patients;
```

Q7 - Show first name, last name, and the **full** province name of each patient.

Example: 'Ontario' instead of 'ON'

```
1 SELECT patients.first_name, patients.last_name, province_names.province_name
2 FROM patients
3 JOIN province_names
4 ON patients.province_id = province_names.province_id;
```

Q8 - Show how many patients have a birth_date with 2010 as the birth year.

```
1 SELECT COUNT(*) FROM patients
2 WHERE YEAR(birth_date) = 2010;
```

Q9 - Show the first_name, last_name, and height of the patient with the greatest height.

```
1 SELECT first_name, last_name, height
2 FROM patients
3 WHERE height = (SELECT MAX(height) FROM patients);
```

Q10 - Show all columns for patients who have one of the following patient_ids:
1,45,534,879,1000

```
1 SELECT * FROM patients
2 WHERE patient_id IN (1,45,534,879,1000);
```

Q11 – Show the total number of admissions

```
1 SELECT COUNT(*) FROM admissions;
```

Q12 - Show all the columns from admissions where the patient was admitted and discharged on the same day.

```
1 SELECT * FROM admissions
2 WHERE admission_date = discharge_date;
```

Q13 - Show the patient id and the total number of admissions for patient_id 579.

```
1 SELECT patient_id, COUNT(*) AS total_admissions
2 FROM admissions
3 WHERE patient_id = 579;
4
```

Q14 - Based on the cities that our patients live in, show unique cities that are in province_id 'NS'.

```
1 SELECT DISTINCT city
2 FROM patients
3 WHERE province_id = 'NS';
4
```

Q15 - Write a query to find the first_name, last name and birth date of patients who has height greater than 160 and weight greater than 70

```
1 SELECT first_name, last_name, birth_date
2 FROM patients
3 WHERE height > 160 AND weight > 70;
4
```

Q16 - Write a query to find list of patients first_name, last_name, and allergies where allergies are not null and are from the city of 'Hamilton'

```
1 SELECT first_name, last_name, allergies
2 FROM patients
3 WHERE allergies IS NOT NULL
4 AND city = 'Hamilton';
5
```

MEDIUM

Q17 - Show unique birth years from patients and order them by ascending.

```
1 SELECT DISTINCT YEAR(birth_date) AS birth_year
2 FROM patients
3 ORDER BY birth_year ASC;
4
```

Q18 - Show unique first names from the patients table which only occurs once in the list.

For example, if two or more people are named 'John' in the first_name column then don't include their name in the output list. If only 1 person is named 'Leo' then include them in the output.

```
1 SELECT first_name
2 FROM patients
3 GROUP BY first_name
4 HAVING COUNT(*) = 1;
5
```

Q19 - Show patient_id and first_name from patients where their first_name start and ends with 's' and is at least 6 characters long.

```
1 SELECT patient_id , first_name
2 FROM patients
3 WHERE first_name LIKE 's____s';
4
```

Q20 - Show patient_id, first_name, last_name from patients whos diagnosis is 'Dementia'.
Primary diagnosis is stored in the admissions table.

```
1 SELECT patients.patient_id, patients.first_name, patients.last_name
2 FROM patients
3 JOIN admissions
4 ON patients.patient_id = admissions.patient_id
5 WHERE admissions.diagnosis = 'Dementia';
6
```

Q21 - Display every patient's first_name.

Order the list by the length of each name and then by alphabetically.

```
1 SELECT first_name FROM patients
2 ORDER BY LENGTH(first_name), first_name;
3
```

Q22 - Show the total amount of male patients and the total amount of female patients in the patients table.

Display the two results in the same row.

```
1 SELECT
2
3     (SELECT COUNT(*) FROM patients WHERE gender = 'M') AS male_count,
4     (SELECT COUNT(*) FROM patients WHERE gender = 'F') AS female_count;
5
```

Q23 - Show first and last name, allergies from patients which have allergies to either 'Penicillin' or 'Morphine'.

Show results ordered ascending by allergies then by first_name then by last_name.

```
1 SELECT first_name, last_name, allergies
2 FROM patients
3 WHERE allergies IN ('Penicillin','Morphine')
4 ORDER BY allergies, first_name, last_name ASC;
5
```

Q24 - Show patient_id, diagnosis from admissions. Find patients admitted multiple times for the same diagnosis.

```
1 SELECT patient_id, diagnosis
2 FROM admissions
3 GROUP BY patient_id, diagnosis
4 HAVING COUNT(*) > 1;
5
```

Q25 - Show the city and the total number of patients in the city. Order from most to least patients and then by city name ascending.

```
1 SELECT city, COUNT(city) AS num_patients
2 FROM patients
3 GROUP BY city
4 ORDER BY num_patients DESC, city ASC;
5
```

Q26 - Show first name, last name and role of every person that is either patient or doctor. The roles are either "Patient" or "Doctor"

```
1 SELECT first_name, last_name, 'Patient' AS role
2 FROM patients
3 UNION ALL
4 SELECT first_name, last_name, 'Doctor' AS role
5 FROM doctors;
```

Q27 - Show all allergies ordered by popularity. Remove NULL values from query.

```
1 SELECT allergies, COUNT(*) AS total_diagnosis
2 FROM patients
3 WHERE allergies IS NOT NULL
4 GROUP BY allergies
5 ORDER BY total_diagnosis DESC;
6
```

Q28 - Show all patient's first_name, last_name, and birth_date who were born in the 1970s decade. Sort the list starting from the earliest birth_date.

```
1 SELECT first_name, last_name, birth_date
2 FROM patients
3 WHERE birth_date BETWEEN '1970-01-01' AND '1979-12-31'
4 ORDER BY birth_date ASC;
```

Q29 - We want to display each patient's full name in a single column. Their last_name in all upper letters must appear first, then first_name in all lower case letters. Separate the last_name and first_name with a comma.

Order the list by the first_name in decending order

EX: SMITH,jane

```
1 SELECT CONCAT(UPPER(last_name),',',LOWER(first_name))
2 FROM patients
3 ORDER BY first_name DESC;
```

Q30 - Show the province_id(s), sum of height; where the total sum of its patient's height is greater than or equal to 7,000.

```
1 SELECT province_id, SUM(height) AS sum_height
2 FROM patients
3 GROUP BY province_id
4 HAVING sum_height > 7000 OR sum_height = 7000;
```

Q31 - Show the difference between the largest weight and smallest weight for patients with the last name 'Maroni'

```
1 SELECT (MAX(weight) - MIN(weight)) AS weight_delta
2 FROM patients
3 WHERE last_name = 'Maroni';
4
```

Q32 - Show all of the days of the month (1-31) and how many admission_dates occurred on that day. Sort by the day with most admissions to least admissions.

```
1 SELECT DAY(admission_date) AS day_number,
2 COUNT(patient_id) AS number_of_admissions
3 FROM admissions
4 GROUP BY day_number
5 ORDER BY number_of_admissions DESC;
```

Q33 - Show all columns for patient_id 542's most recent admission_date.

```
1 SELECT * FROM admissions
2 WHERE patient_id = 542 AND
3 admission_date = (SELECT MAX(admission_date) FROM admissions WHERE patient_id = 542);
```

Q34 - Show patient_id, attending_doctor_id, and diagnosis for admissions that match one of the two criteria:

1. patient_id is an odd number and attending_doctor_id is either 1, 5, or 19.
2. attending_doctor_id contains a 2 and the length of patient_id is 3 characters.

```
1 SELECT patient_id, attending_doctor_id, diagnosis
2 FROM admissions
3 WHERE
4 ((patient_id % 2 <> 0) AND (attending_doctor_id IN (1,5,19)))
5 OR
6 ((attending_doctor_id LIKE '%2%') AND (LEN(patient_id) = 3));
```

Q35 - Show first_name, last_name, and the total number of admissions attended for each doctor.

Every admission has been attended by a doctor.

```
1 SELECT doctors.first_name, doctors.last_name,
2 COUNT(admissions.attending_doctor_id) AS admissions_total
3 FROM doctors
4 JOIN admissions
5 ON doctors.doctor_id = admissions.attending_doctor_id
6 GROUP BY admissions.attending_doctor_id;
```

Q36 - For each doctor, display their id, full name, and the first and last admission date they attended.

```
1 SELECT doctors.doctor_id, CONCAT(doctors.first_name, ' ', doctors.last_name) AS full_name,
2 MIN(admissions.admission_date) AS first_admission_date,
3 MAX(admissions.admission_date) AS last_admission_date
4 FROM doctors
5 JOIN admissions
6 ON doctors.doctor_id = admissions.attending_doctor_id
7 GROUP BY doctors.doctor_id;
```

Q37 - Display the total amount of patients for each province. Order by descending.

```
1 SELECT province_names.province_name, COUNT(patients.patient_id) AS patient_count
2 FROM patients
3 JOIN province_names
4 ON patients.province_id = province_names.province_id
5 GROUP BY province_names.province_name
6 ORDER BY patient_count DESC;
```

Q38 - For every admission, display the patient's full name, their admission diagnosis, and their doctor's full name who diagnosed their problem.

```
1 SELECT
2 CONCAT(patients.first_name, ' ', patients.last_name),
3 admissions.diagnosis,
4 CONCAT(doctors.first_name, ' ', doctors.last_name)
5 FROM patients
6 JOIN admissions ON patients.patient_id = admissions.patient_id
7 JOIN doctors ON admissions.attending_doctor_id = doctors.doctor_id;
```

Q39 - display the first name, last name and number of duplicate patients based on their first name and last name. Ex: A patient with an identical name can be considered a duplicate.

```
1 SELECT first_name, last_name, COUNT(*) AS num_of_duplicates
2 FROM patients
3 GROUP BY first_name, last_name
4 HAVING num_of_duplicates > 1;
```

Q40 - Display patient's full name, height in the units feet rounded to 1 decimal, weight in the unit pounds rounded to 0 decimals, birth_date, gender non abbreviated.

Convert CM to feet by dividing by 30.48.

Convert KG to pounds by multiplying by 2.205.

```
1 SELECT CONCAT(first_name, ' ', last_name) AS patient_name,
2 ROUND(height/30.48,1) AS "height_feet",
3 ROUND(weight*2.205,0) AS "weight_pounds",
4 birth_date,
5 CASE gender
6     WHEN 'M' THEN 'MALE'
7     ELSE 'FEMALE'
8 END AS gender_type
9 FROM patients;
```

Q41 - Show patient_id, first_name, last_name from patients whose does not have any records in the admissions table. (Their patient_id does not exist in any admissions.patient_id rows.)

```
1 SELECT patient_id, first_name, last_name
2 FROM patients
3 WHERE patient_id NOT IN (SELECT patient_id FROM admissions);
```

Q42 - Display a single row with max_visits, min_visits, average_visits where the maximum, minimum and average number of admissions per day is calculated. Average is rounded to 2 decimal places.

```
1 SELECT MAX(daily_admission) AS max_visits,
2 MIN(daily_admission) AS min_visits,
3 ROUND(AVG(daily_admission),2) AS average_visits
4 FROM
5 (SELECT admission_date, COUNT(*) AS daily_admission
6 FROM admissions
7 GROUP BY admission_date);
```

Q43 - Display every patient that has at least one admission and show their most recent admission along with the patient and doctor's full name.

```
1 SELECT CONCAT(patients.first_name, ' ', patients.last_name) AS patient_name,
2 MAX(admissions.admission_date) AS admission_date,
3 CONCAT(doctors.first_name, ' ', doctors.last_name) AS doctor_name
4 FROM patients
5 JOIN admissions ON patients.patient_id = admissions.patient_id
6 JOIN doctors ON admissions.attending_doctor_id = doctors.doctor_id
7 WHERE admissions.admission_date = (
8     SELECT MAX(admissions.admission_date)
9     FROM admissions
10    WHERE admissions.patient_id = patients.patient_id
11 );
```

HARD

Q44 - Show all of the patients grouped into weight groups.
Show the total amount of patients in each weight group.
Order the list by the weight group descending.

For example, if they weight 100 to 109 they are placed in the 100 weight group, 110-119 = 110 weight group, etc.

```
1 SELECT
2     (weight / 10) * 10 AS weight_group,
3     COUNT(*) AS patients_in_group
4 FROM patients
5 GROUP BY weight_group
6 ORDER BY weight_group DESC;
```

Q45 - Show patient_id, weight, height, isObese from the patients table. Display isObese as a boolean 0 or 1. Obese is defined as $\text{weight(kg)} / (\text{height(m)}^2) \geq 30$. weight is in units kg. height is in units cm.

```
1 SELECT patient_id, weight, height,
2 CASE
3     WHEN (weight*10000/(height*height)) >= 30 THEN 1
4     ELSE 0
5 END AS isObese
6 FROM patients;
```

Q46 - Show patient_id, first_name, last_name, and attending doctor's specialty.
Show only the patients who has a diagnosis as 'Epilepsy' and the doctor's first name is 'Lisa'
Check patients, admissions, and doctors tables for required information.

```
1 SELECT patients.patient_id, patients.first_name, patients.last_name,
2 doctors.specialty
3 FROM patients
4 JOIN admissions ON patients.patient_id = admissions.patient_id
5 JOIN doctors ON admissions.attending_doctor_id = doctors.doctor_id
6 WHERE admissions.diagnosis = 'Epilepsy' AND doctors.first_name = 'Lisa';
```

Q47 - All patients who have gone through admissions, can see their medical documents on our site. Those patients are given a temporary password after their first admission. Show the patient_id and temp_password. The password must be the following, in order:

1. patient_id
2. the numerical length of patient's last_name
3. year of patient's birth_date

```
1 SELECT patients.patient_id,
2 CONCAT(patients.patient_id, LEN(patients.last_name), YEAR(patients.birth_date)) AS temp_password
3 FROM patients
4 JOIN admissions ON patients.patient_id = admissions.patient_id
5 GROUP BY patients.patient_id, patients.last_name, patients.birth_date;
```

Q48 - Each admission costs \$50 for patients without insurance, and \$10 for patients with insurance. All patients with an even patient_id have insurance.
Give each patient a 'Yes' if they have insurance, and a 'No' if they don't have insurance. Add up the admission_total cost for each has_insurance group.

```

1 SELECT
2     CASE
3         WHEN patient_id % 2 == 0 THEN 'Yes'
4         ELSE 'No'
5     END AS has_insurance,
6
7     CASE
8         WHEN patient_id % 2 == 0 THEN (COUNT(*) * 10)
9         ELSE (COUNT(*) * 50)
10    END AS cost_after_insurance
11 FROM admissions
12 GROUP BY has_insurance;

```

Q49 - Show the provinces that has more patients identified as 'M' than 'F'. Must only show full province_name

```

1 SELECT province_names.province_name
2 FROM province_names
3 JOIN patients ON patients.province_id = province_names.province_id
4 GROUP BY province_names.province_name
5 HAVING
6     SUM(CASE WHEN patients.gender = 'M' THEN 1 ELSE 0 END)
7     > SUM(CASE WHEN patients.gender = 'F' THEN 1 ELSE 0 END);

```

Q50 - We are looking for a specific patient. Pull all columns for the patient who matches the following criteria:

- First_name contains an 'r' after the first two letters.
- Identifies their gender as 'F'
- Born in February, May, or December
- Their weight would be between 60kg and 80kg
- Their patient_id is an odd number
- They are from the city 'Kingston'

```

1 SELECT * FROM patients
2 WHERE
3 first_name LIKE '__r%' AND
4 gender = 'F' AND
5 MONTH(birth_date) IN (02,05,12) AND
6 (weight BETWEEN 60 AND 80) AND
7 city = 'Kingston' AND
8 patient_id % 2 != 0;

```

Q51 - Show the percent of patients that have 'M' as their gender. Round the answer to the nearest hundredth number and in percent form.

```

1 SELECT CONCAT(ROUND((SUM(CASE WHEN gender = 'M' THEN 1.0 ELSE 0.0 END) / COUNT(*)) * 100, 2
2 AS percent_m
3 FROM patients;
4

```

```

1 CONCAT(ROUND((SUM(CASE WHEN gender = 'M' THEN 1.0 ELSE 0.0 END) / COUNT(*)) * 100, 2), '%')
2 ent_m
3 tients;
4

```

Q52 - For each day display the total amount of admissions on that day. Display the amount changed from the previous date.

```
1 SELECT admission_date, COUNT(admission_date) AS admission_day,  
2 COUNT(admission_date) - LAG(COUNT(admission_date)) OVER (ORDER BY admission_date)  
3 AS admission_count_change  
4 FROM admissions  
5 GROUP BY admission_date;
```

-----[NEW CONCEPT LEARNED – LAG()]-----

Q53 - Sort the province names in ascending order in such a way that the province 'Ontario' is always on top.

```
1 SELECT province_name FROM province_names  
2 ORDER BY  
3 CASE WHEN province_name = 'Ontario' THEN 0 ELSE 1 END ASC,  
4 province_name ASC;
```

[NOTE – Read explanation of this query, good concept]

Q54 - We need a breakdown for the total amount of admissions each doctor has started each year. Show the doctor_id, doctor_full_name, specialty, year, total_admissions for that year.

```
1 SELECT doctors.doctor_id, CONCAT(doctors.first_name," ",doctors.last_name)  
2 AS doctor_name,  
3 doctors.specialty, YEAR(admissions.admission_date) AS selected_year,  
4 COUNT(admissions.attending_doctor_id) AS total_admissions  
5 FROM doctors  
6 JOIN admissions ON admissions.attending_doctor_id = doctors.doctor_id  
7 GROUP BY  
8 admissions.attending_doctor_id,  
9 YEAR(admissions.admission_date);
```